Laurent Ortiz

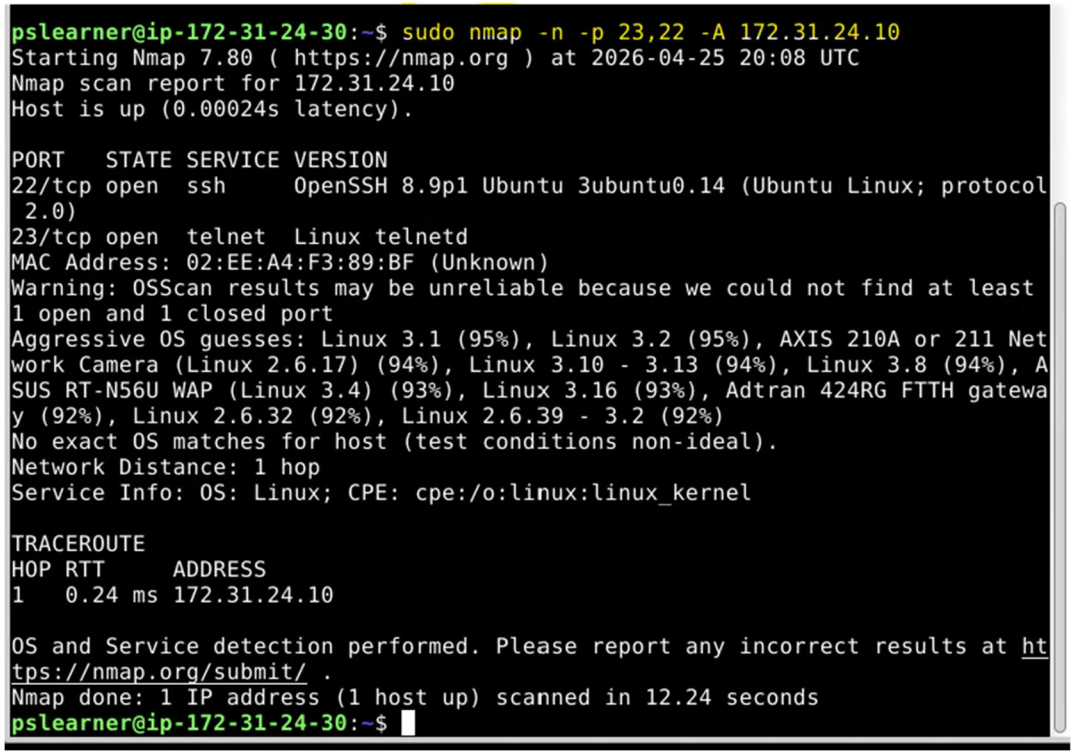
CYBR 4450

M14: Final Project

4/25/26

Step 1.

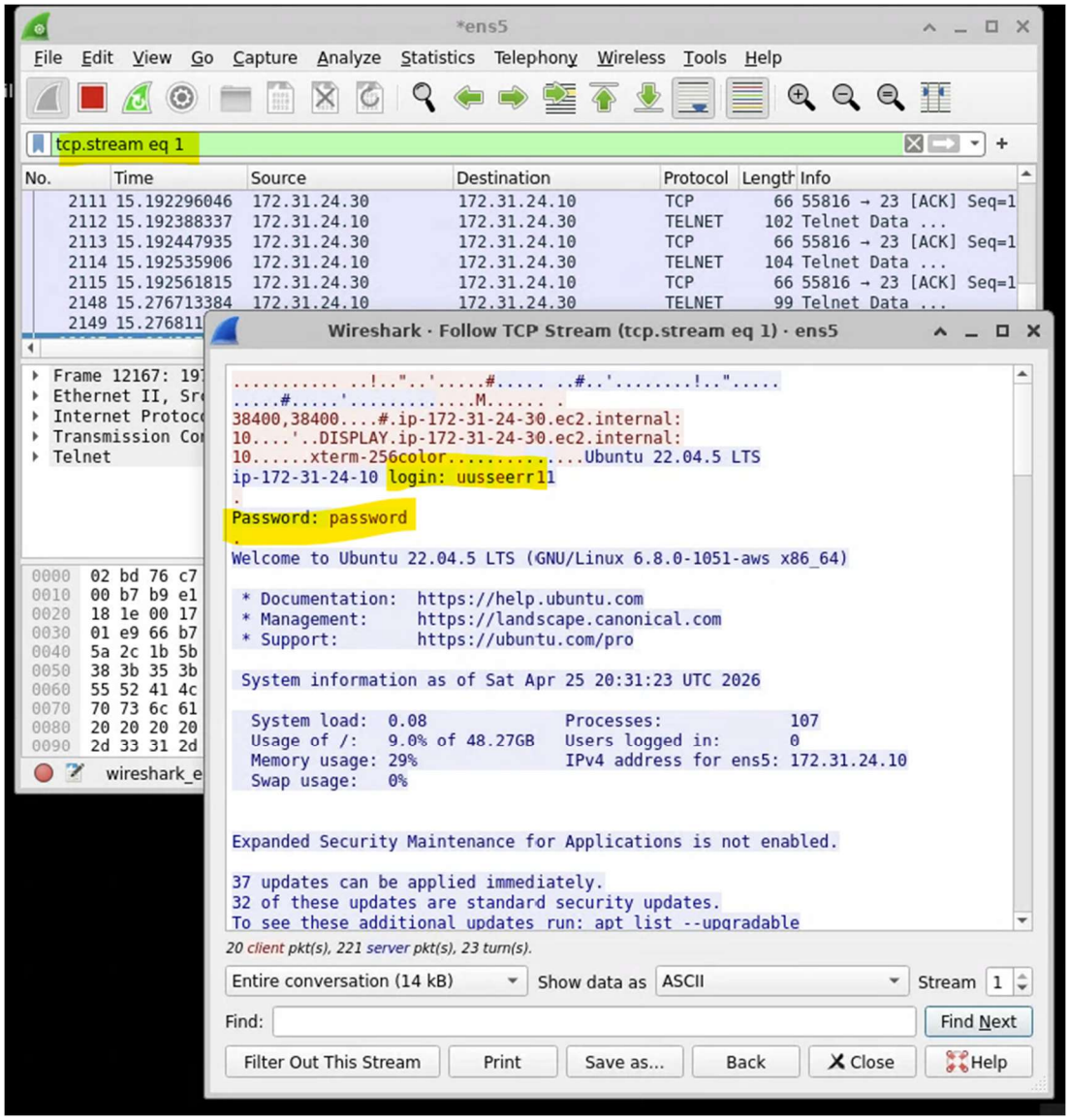
The nmap command scanned the target, 172.31.24.10, for open ports. The scan finished by identifying port 22 (SSH) and port 23 (Telnet). Port 22 is running OpenSSH8.9p1 on a Ubuntu system, which is being used for secure remote administration, but can be exploitable if the credentials are weak. Since port 23 is running a Telnet service, it's transmitting data like login credentials in plaintext, making a great target for us to exploit.



Step 2.

Telnet Capture:

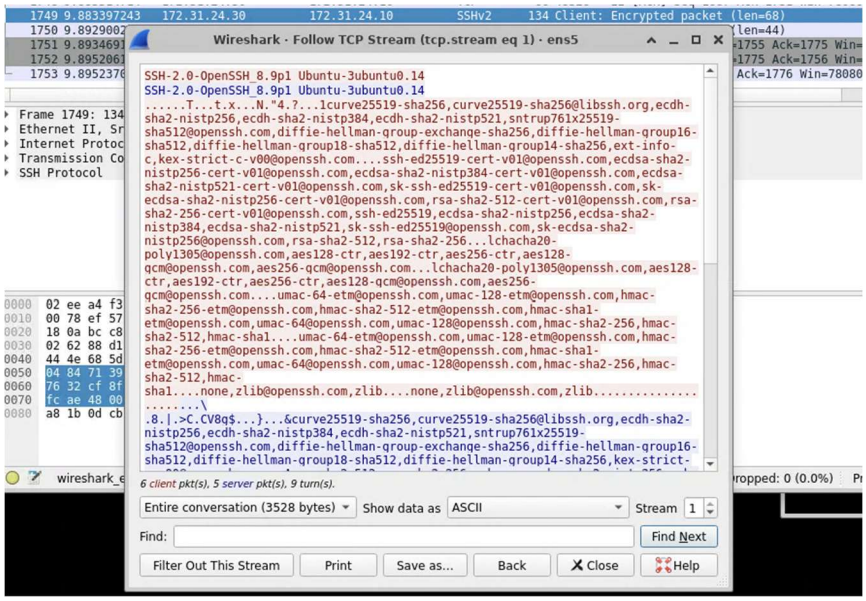
In my terminal, I opened a Telnet session to the target system, 172.31.24.10. In Wireshark, I filtered the traffic by using *tcp.port == 23*. I opened one of the packets by selecting Follow TCP Stream and I was able to view the plaintext credentials, username: user1 and Password: password. These are very weak credentials and shows how Telnet is highly insecure since we can easily intercept the sensitive information.



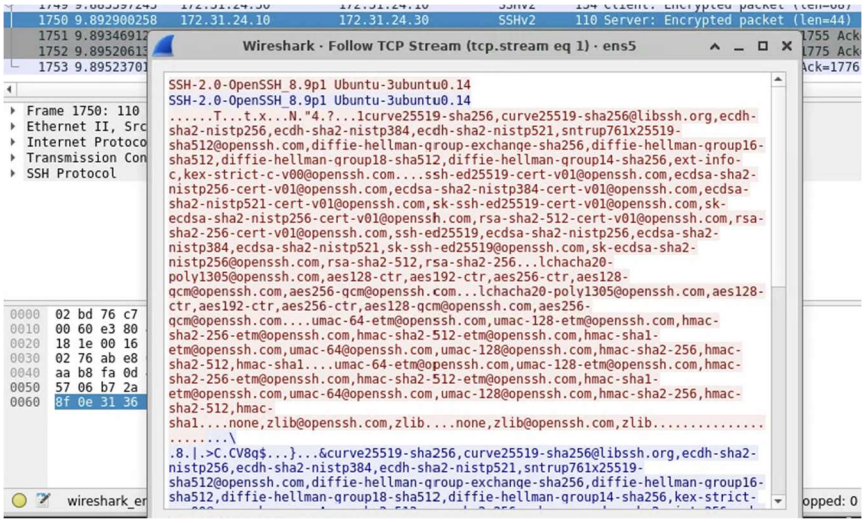
Screenshot of Wireshark Telnet Packet- Shows plaintext credentials by following TCP stream.

SSH Capture:

I started a new Wireshark capture to view SSH packets by entering the command *ssh user1@172.31.24.10* to open a SSH session. During the SSH capture, I filtered the traffic using *tcp.port == 22*. When analyzing the packets and following the TCP stream, the data appeared encrypted and unreadable. Unlike Telnet, no login credentials were visible in plaintext. Both Client and Server packets from the screenshots below were labeled as “Encrypted Packet”, and the output was random binary and hexadecimal data. This shows that SSH encrypts communication, and protects data from interception from attackers.



Screenshot of SSH Client Packet – No plaintext data visible, only encrypted data.



Screenshot of SSH Server Packet – No plaintext data visible, only encrypted data.

Step 3.

Telnet Brute Force:

Using Metasploit, I performed a Telnet brute-force attack on the target system. I loaded into the telnet login module to setup usernames and password lists to run the attack. After several tries, I had two successful logins from the command: user1:password and user2:password1. After running this command, it shows that the Telnet service is highly vulnerable to brute-force attacks due to weak password usage and lack of authentication security.

```
msf auxiliary(scanner/telnet/telnet_login) > set RHOSTS 172.31.24.10
RHOSTS => 172.31.24.10
msf auxiliary(scanner/telnet/telnet_login) > set USER_FILE ~/usernames.lst
USER_FILE => ~/usernames.lst
msf auxiliary(scanner/telnet/telnet_login) > set PASS_FILE ~/passwords.lst
PASS_FILE => ~/passwords.lst
```

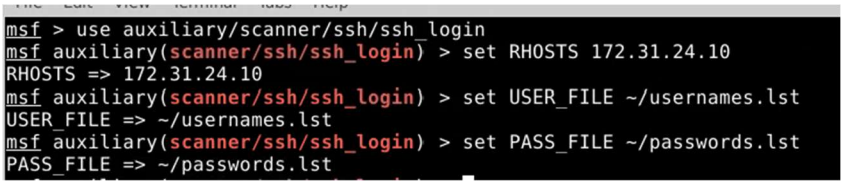
Screenshot of Metasploit commands used to brute-force the Telnet service for login credentials.

```
File  Edit  View  Terminal  Tabs  Help
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user1:toor (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user1:rootpw (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user1:pass (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user1:passpw (Incorrect: )
[+] 172.31.24.10:23       - 172.31.24.10:23 - Login Successful: user1:password
[*] 172.31.24.10:23       - Attempting to start session 172.31.24.10:23 with user1:passw
ord
[*] Command shell session 1 opened (172.31.24.30:38243 -> 172.31.24.10:23) at 2026-04-25
 21:37:11 +0000
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:toor (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:rootpw (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:pass (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:passpw (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:password (Incorrect: )
[-] 172.31.24.10:23       - 172.31.24.10:23 - LOGIN FAILED: user2:passwordone (Incorrect
: )
[+] 172.31.24.10:23       - 172.31.24.10:23 - Login Successful: user2:password1
[*] 172.31.24.10:23       - Attempting to start session 172.31.24.10:23 with user2:passw
ord1
```

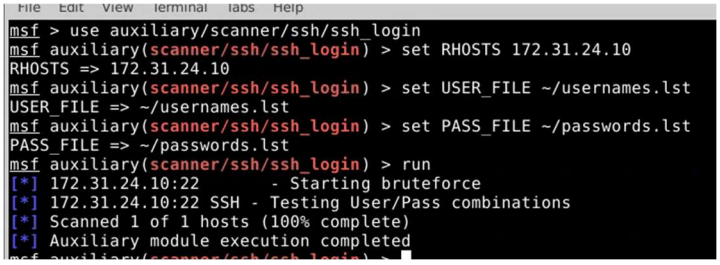
Screenshot of after the Telnet brute-force attack, there were 2 successful logins found.

SSH Brute Force:

I performed an SSH brute-forcing attack in Metasploit, but it didn't result in any successful login attempts. Using the same lists from the Telnet attack, it didn't find any valid credentials. This can mean that the SSH service could be configured with stronger authentication protocols. Even though SSH encrypts all communication, encryption alone doesn't prevent brute-force attacks. If weak credentials were used and password authentication were enabled, attackers could still gain access through repeated login attempts. To verify the authentication method in SSH, I attempted to force login credentials using *ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no user1@172.31.24.10*, but it returned with "Permission denied (publickey)." SSH provides confidentiality and with proper authentication configuration, like using a public key, it can greatly improve security.



Screenshot of Metasploit commands used to brute-force the SSH service for login credentials.



Screenshot of after the SSH brute-force attack, there were no successful login attempts.



Screenshot of attempting to force password-based login with user1 under SSH – permission denied.

Step 4.

Reflection:

The results of the Final challenge project showed me that Telnet and SSH are very different when it comes to security, especially when it comes to how they safeguard data and verify users. Telnet was found to be highly insecure since it transmits credentials in plaintext. This makes it very easy for attackers to compromise systems using brute-force attacks, resulting in successful logins. And since the credentials were very weak, it made it much easier and faster to exploit. Compared to SSH, where it encrypts all communication preventing attackers from viewing and intercepting sensitive data. The SSH brute-force attempt also showed that it can still be targeted by these attacks in general, but it can be protected by the use of public-key based authentication.

Overall, the outcome from the attacks and packet analysis for both Telnet and SSH shows how important it is to use strong authentication methods in IoT system environments. Weak passwords make it much easier for attackers to gain unauthorized access, while key-based authentication provides a more secure alternative by removing reliance on easily remembered credentials. To improve IoT security, I would suggest disabling Telnet and replacing it with SSH using a public key, to encrypt all data that is being transmitted. Enforcing stronger credential policies or using multifactor authentication can also greatly reduce data breaches and unauthorized access. And another security practice that can be implemented is limiting login attempts and monitoring for suspicious activity can reduce the risk of brute-force attacks.